

AGENT KIT BY ↳ INNGEST

≡ Get Started > Quick start

Get Started

Quick start

Learn the basics of AgentKit in a few minutes.

In this tutorial, you will create an **Agent** and run it within a **Network** using AgentKit.

 Follow this guide by forking the **quick-start** example locally by running:

```
npx git-ripper https://github.com/inngest/agent-kit/tree/main/examples/qu
```

Creating a single agent

1 Install AgentKit

Within an existing project, install AgentKit and Inngest from npm:

```
npm pnpm yarn
```

```
npm install @inngest/agent-kit inngest
```

 **Important:** Starting with AgentKit v0.9.0, `inngest` is a required peer dependency. You must install both packages together to ensure proper runtime compatibility and prevent conflicts.

AGENT KIT BY ↳ INNGEST

>

2 Create an agent

To start, we'll create our first "**Agent**." An Agent is an entity that has a specific role to answer questions or perform tasks (see "tools" below).

Let's create a new file, `index.ts`. Using the `createAgent` constructor, give your agent a `name`, a `description`, and its initial `system` prompt. The `name` and `description` properties are used to help the LLM determine which Agent to call.

You'll also specify which `model` you want the agent to use. Here we'll use Anthropic's **Claude 3.5 Haiku** model. (**Model reference**)

Your agent can be whatever you want, but in this quick start, we'll create a PostgreSQL database administrator agent:

index.ts



```
import { createAgent, anthropic } from '@inngest/agent-kit';

const dbaAgent = createAgent({
  name: 'Database administrator',
  description: 'Provides expert support for managing PostgreSQL databases',
  system:
    'You are a PostgreSQL expert database administrator. ' +
    'You only provide answers to questions related to PostgreSQL database s
  model: anthropic({
    model: 'claude-3-5-haiku-latest',
    defaultParameters: {
      max_tokens: 1000,
    },
  }),
});
```

AGENT KIT BY ↳ INNGEST

>

3 Run the server

Next, we'll create an HTTP server to run our agent. In the same file as our Agent definition:

index.ts

```
import { createAgent, anthropic } from '@inngest/agent-kit';
import { createServer } from '@inngest/agent-kit/server';
// ...
const server = createServer({
  agents: [dbaAgent],
});
server.listen(3000, () => console.log('AgentKit server running!'));
```

Now we can run our AgentKit server using `npx` and `tsx` (for easy TypeScript execution):

terminal

```
npx tsx ./index.ts
```

4 Test our agent

To test our agent, we'll use the [Inngest dev server](#) to visually debug our agents. Using `npx`, we'll start the server and point it to our AgentKit server:

AGENT KIT BY ↳ INNGEST

>

Now, open the dev server and select the functions tab

(`http://localhost:8288/functions`) and click the “Invoke” button:

In the Invoke function modal, specify the input prompt for your agent and click the “Invoke function” button:

AGENT KIT BY ↳ INNGEST

>

Invoke payload



```
{  
  "data": {  
    "input": "How do I aggregate an integer column across a date column by v  
  }  
}
```

You'll be redirected to watch the agent run and view the output:

AGENT KIT BY ↳ INNGEST

>

A key benefit of AgentKit is the ability to create a system of agents called a "**Network.**" Networks are used to create AI Agents by combining multiple specialized **Agents** to answer more complex questions. Let's transform our single agent into a network of two agents, capable of helping with both database administration and security questions.

Creating a multi-agent network

1 Adding a second Agent

Agents collaborate in a Network by sharing a common **State.**

Let's update our Database Administrator Agent to include a tool to save the answer to the question in the database:

AGENT KIT BY ↳ INNGEST

>

```

    "You are a PostgreSQL expert database administrator. " +
    "You only provide answers to questions related to PostgreSQL database sc

model: anthropic({
  model: "claude-3-5-haiku-latest",
  defaultParameters: {
    max_tokens: 4096,
  },
}),
tools: [
  createTool({
    name: "save_answer",
    description: "Save the answer to the questions",
    parameters: z.object({
      answer: z.string(),
    }),
    handler: async ({ answer }, { network }: Tool.Options<NetworkState>) =
      network.state.data.db_agent_answer = answer;
  },
),
],
});

```

 **Tools** are based on **Tool Calling**, enabling your Agent to interact with the **State** of the Network, store data in external databases, or dynamically fetch data from third-party APIs.

Let's now create a second *Database Security* Agent:

AGENT KIT BY ↳ INNGEST

>

```
const securityAgent = createAgent({
  name: "Database Security Expert",
  description:
    "Provides expert guidance on PostgreSQL security, access control, audit system:
    \"You are a PostgreSQL security expert. \" +
    \"You only provide answers to questions related to PostgreSQL security to
  model: anthropic({
    model: "claude-3-5-haiku-latest",
    defaultParameters: {
      max_tokens: 1000,
    },
  }),
  tools: [
    createTool({
      name: "save_answer",
      description: "Save the answer to the questions",
      parameters: z.object({
        answer: z.string(),
      }),
      handler: async ({ answer }, { network }: Tool.Options<NetworkState>) =
        network.state.data.security_agent_answer = answer;
    },
  ),
],
});
```

Our second Security Expert Agent is similar to the first, but with a different system prompt specifically for security questions.

We can now create a network combining our "Database Administrator" and "Database Security" Agents, which enables us to answer more complex questions.

2 Creating a Network

AGENT KIT BY ↳ INNGEST

>

to can.

```
import { /*...*/ createNetwork } from "@inngest/agent-kit";

export interface NetworkState {
  // answer from the Database Administrator Agent
  dba_agent_answer?: string;

  // answer from the Security Expert Agent
  security_agent_answer?: string;
}

// ...
const devOpsNetwork = createNetwork<NetworkState>({
  name: "DevOps team",
  agents: [dbaAgent, securityAgent],
  router: async ({ network }) => {
    if (!network.state.data.security_agent_answer) {
      return securityAgent;
    } else if (
      network.state.data.security_agent_answer &&
      network.state.data.dba_agent_answer
    ) {
      return;
    }
    return dbaAgent;
  },
});

const server = createServer({
  agents: [dbaAgent, securityAgent],
  networks: [devOpsNetwork],
});
```



AGENT KIT BY ↳ INNGEST

>

example, we call the Database Administrator Agent followed by the Security Expert Agent before ending the network (by returning `undefined`).

3 Test our network

We'll use the same approach to test our network as we did above.

With your Inngest dev server running, open the dev server and select the functions tab (<http://localhost:8288/functions>) and click the "Invoke" button of the *DevOps team* function with the following payload:

Invoke payload



```
{
  "data": {
    "input": "I am building a Finance application. Help me answer the follow
  }
}
```

The network will now run through the Agents to answer the questions:

AGENT KIT BY ↳ INNGEST

>

Next steps

Congratulations! You've now created your first AI Agent with AgentKit.

In this guide, you've learned that:

Agents are the building blocks of AgentKit. They are used to call a single model to answer specific questions or perform tasks.

Networks are groups of agents that can work together to achieve more complex goals.

Routers, combined with **State**, enable you to control the flow of your Agents.

The following guides will help you build more advanced AI Agents:

Adding Tools to Agents

Let your Agent act and gather data with tools

Implementing reasoning-based routing

Learn how to dynamically route between agents

AGENT KIT BY ↳ INNGEST

>

Support Agent with "Human in the loop"

This AgentKit example shows how to build a Support Agent Network with a "Human in the loop" pattern.

AgentKit SWE-bench

This AgentKit example uses the SWE-bench dataset to train an agent to solve coding problems. It uses advanced tools to interact with files and codebases.

AgentKit

[< Previous](#)

Installation

[Next >](#)

Powered by Mintlify